



Projet Data: Sauvetage Maritime

SIMPLON
.CO

Formation : Data Engineer P1 – 2025–2027

Présenté par :

Caroline BERGE, Kaouter RHAZLANI, Zoubir MABED

Le 16 Janvier 2026

Plan de Présentation

Découvrez le parcours de notre exploration sur le sauvetage maritime.

01

Introduction :

contexte projet / besoin métier

02

Modèle conceptuel et modèle logique de données

03

Pipeline ETL

04

Ingestion

05

Nettoyage et validation (pandera)

06

Chargement en BD

07

Test unitaires

08

Mise en place de l'application streamlit

09

Demo (dashboard / interactions avec la base de données / audit & admin)

10

Conclusion & Perspectives

01 : Introduction



Contexte du projet

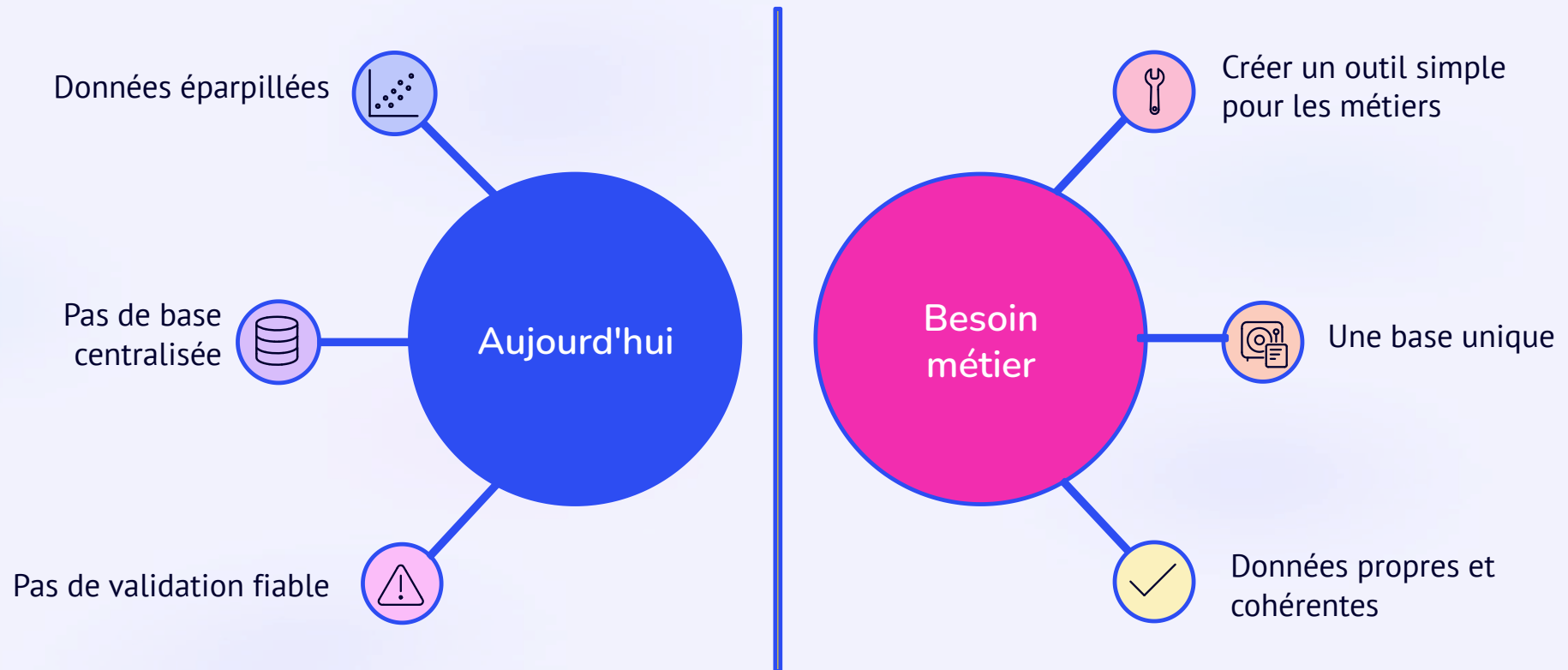
Le **CROSS** (Centre Régional Opérationnel de Surveillance et de Sauvetage) :

- Il coordonne les opérations de sauvetage maritime.
- Il suit :
 - les opérations
 - les navires / flotteurs
 - le bilan humain
 - les statistiques temporelles et géographiques et les statistiques liées à chaque opération

Problématique : Le service a perdu son système, mais a retrouvé des sauvegardes CSV / parquet.

Contexte du projet

Le besoin métier : reconstruire l'historique, le fiabiliser, et le rendre exploitable



Notre objectif :

Construire un pipeline de données complet, fiable et maintenable, de la donnée brute à l'outil métier

Gestion de projet :

Brief 3 : Mise en place d'une d'une solution de gestion de données et d'une interface analytique polyvalente...

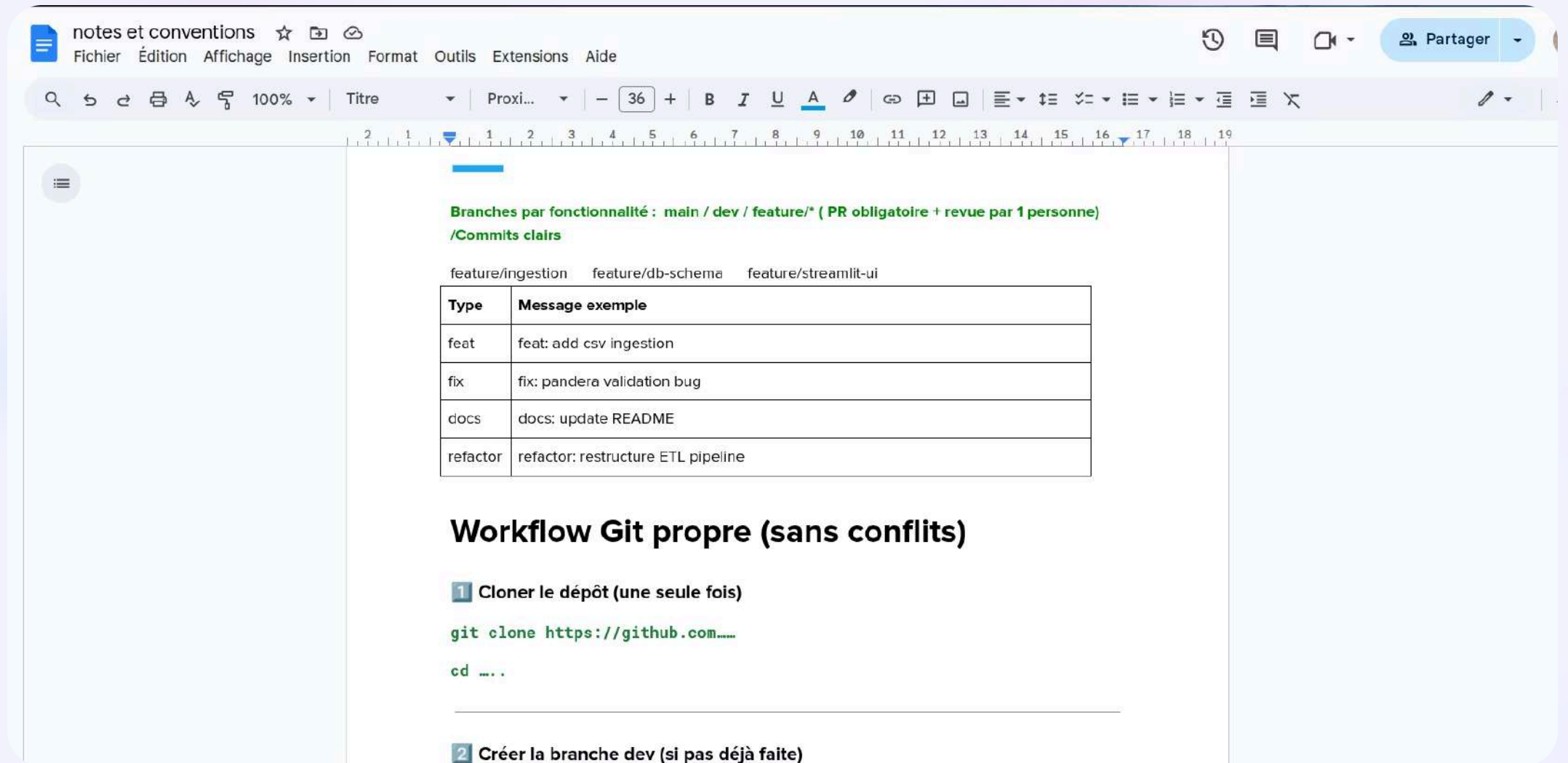
Fichier Édition Affichage Insertion Format Données Outils Extensions Aide

100% 123 Roboto 14 B I A

A2 fx 0.Pilotage & organisation

Tâche	Priorité	Propriétaire	État	Date de début	Date de fin	Livrable	Notes
0.Pilotage & organisation	P0	Nom	Terminé	12/01/2026	12/01/2026	Fichier	Notes
Désigner le chef de projet	P0	Kaouter RHA...	Terminé	12/01/2026	12/01/2026	Fichier	Notes
Définir les rôles (Ingestion / DB / Analytics / Streamlit)	P0	mabedzoubir...	Terminé	12/01/2026	12/01/2026	https://github.com/Simplon-	Notes
Créer le repo Git (voir structure initiale dans le docs des notes partagé)	P0	mabedzoubir...	Terminé	12/01/2026	12/01/2026	Fichier	Notes
Définir conventions de nommage (branches, dossiers, variables)	P0	Kaouter RHA...	Terminé	12/01/2026	12/01/2026	notes et conventions	Notes
Mettre en place un tableau Kanban	P0	Kaouter RHA...	Terminé	12/01/2026	12/01/2026	Fichier	Notes
Planifier les réunions d'avancement	P0	Kaouter RHA...	Terminé	12/01/2026	12/01/2026	Fichier	Notes
Définir planning global	P0	Kaouter RHA...	Terminé	12/01/2026	12/01/2026	Fichier	Notes
Livrable : repo / Structure initiale en place / Conventions Git définies	P0	Kaouter RHA...	Terminé	12/01/2026	12/01/2026	Fichier	Notes
1.Compréhension métier & données	P0	Nom	Terminé	12/01/2026	14/01/2026	Fichier	Notes
1-1/Lire la documentation officielle des données	P0	grass squad	Terminé	12/01/2026	dd/mm/yyyy	Fichier	Notes
doc datagouv / fichiers dispo / metadonnées / date debut et fin pour chaq fichiers	P0	caroline.berge31@	En cours	12/01/2026	dd/mm/yyyy	Fichier	Notes
1-2/Identifier les entités métier principales	P0	caroline.berge31@	En cours	12/01/2026	dd/mm/yyyy	Fichier	Notes
interventions /alerte / moyens engagés/floteurs/bilan humain/météo / zone géo	P0	grass squad	En cours	12/01/2026	dd/mm/yyyy	Fichier	Notes

Notes & conventions de travail sur git



notes et conventions ☆ 📁 ☁

Fichier Édition Affichage Insertion Format Outils Extensions Aide

Titre Proxi... 36 B I U A

Branches par fonctionnalité : main / dev / feature/* (PR obligatoire + revue par 1 personne)
/Commits clairs

feature/ingestion feature/db-schema feature/streamlit-ui

Type	Message exemple
feat	feat: add csv ingestion
fix	fix: pandera validation bug
docs	docs: update README
refactor	refactor: restructure ETL pipeline

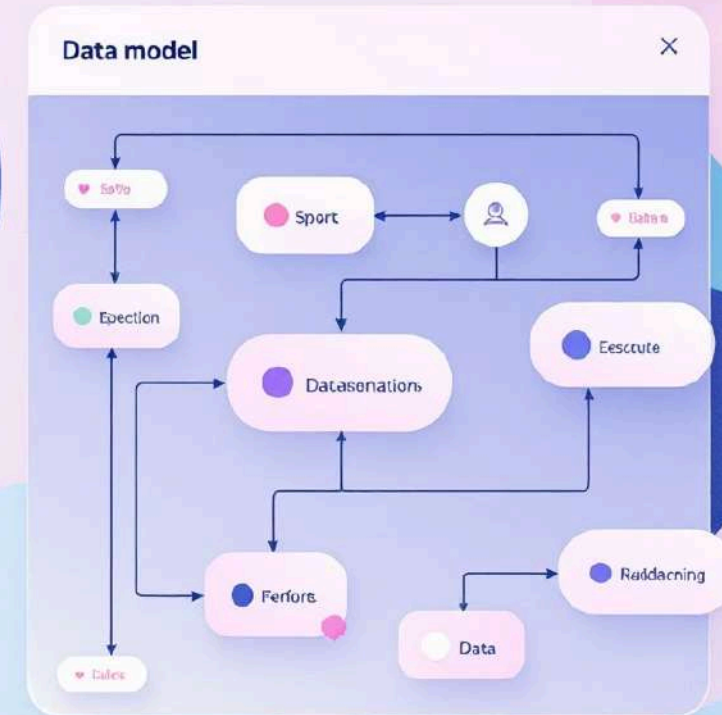
Workflow Git propre (sans conflits)

- 1 Cloner le dépôt (une seule fois)

```
git clone https://github.com.....
```

```
cd ....
```
- 2 Créer la branche dev (si pas déjà faite)

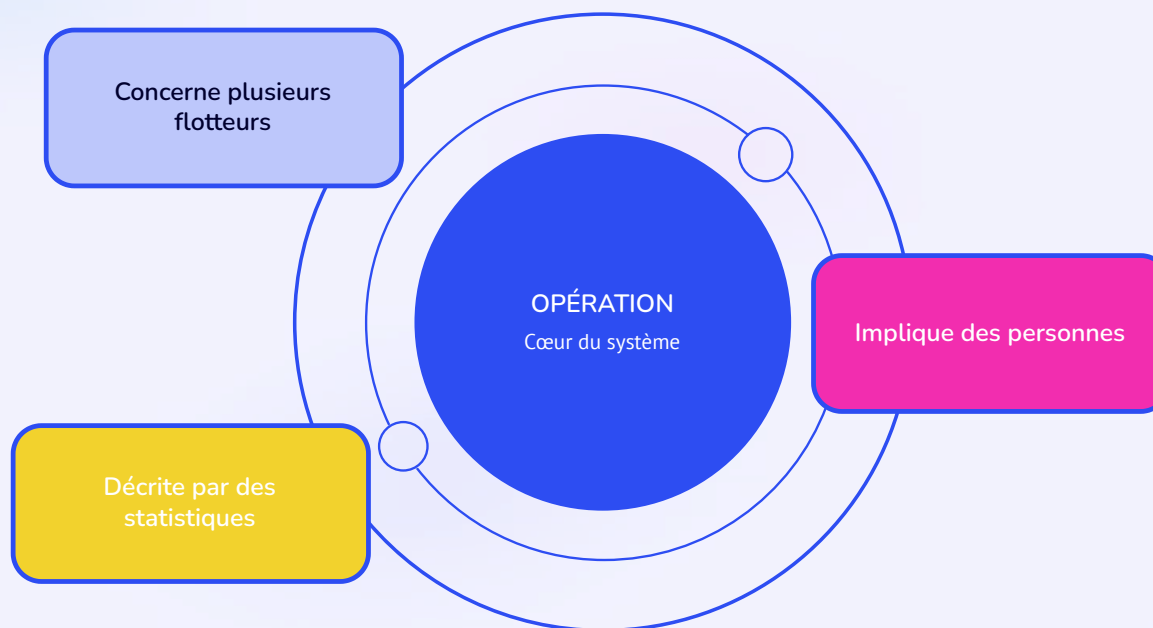
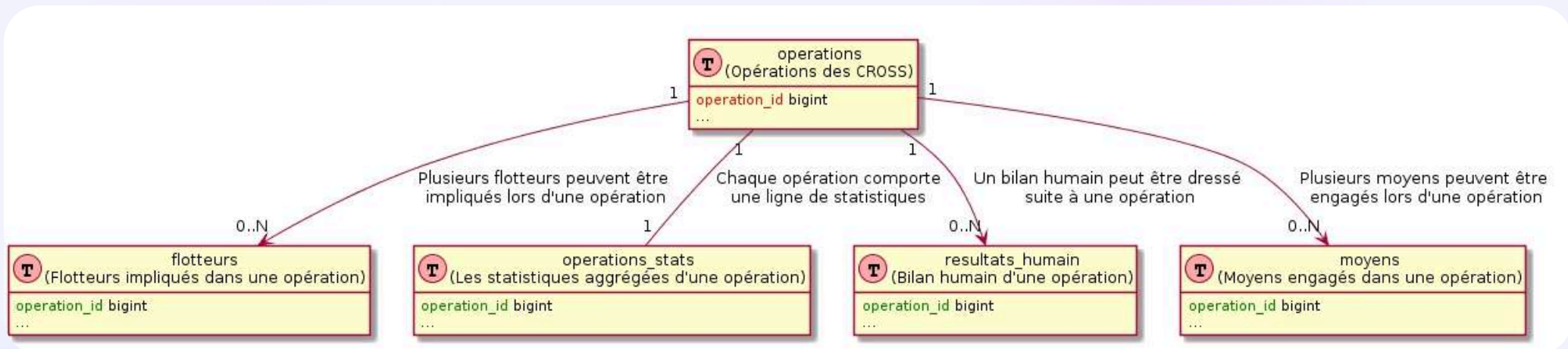
02 : Modèle conceptuel et modèle logique de données



Modèle conceptuel de données

Fichiers CSV disponible

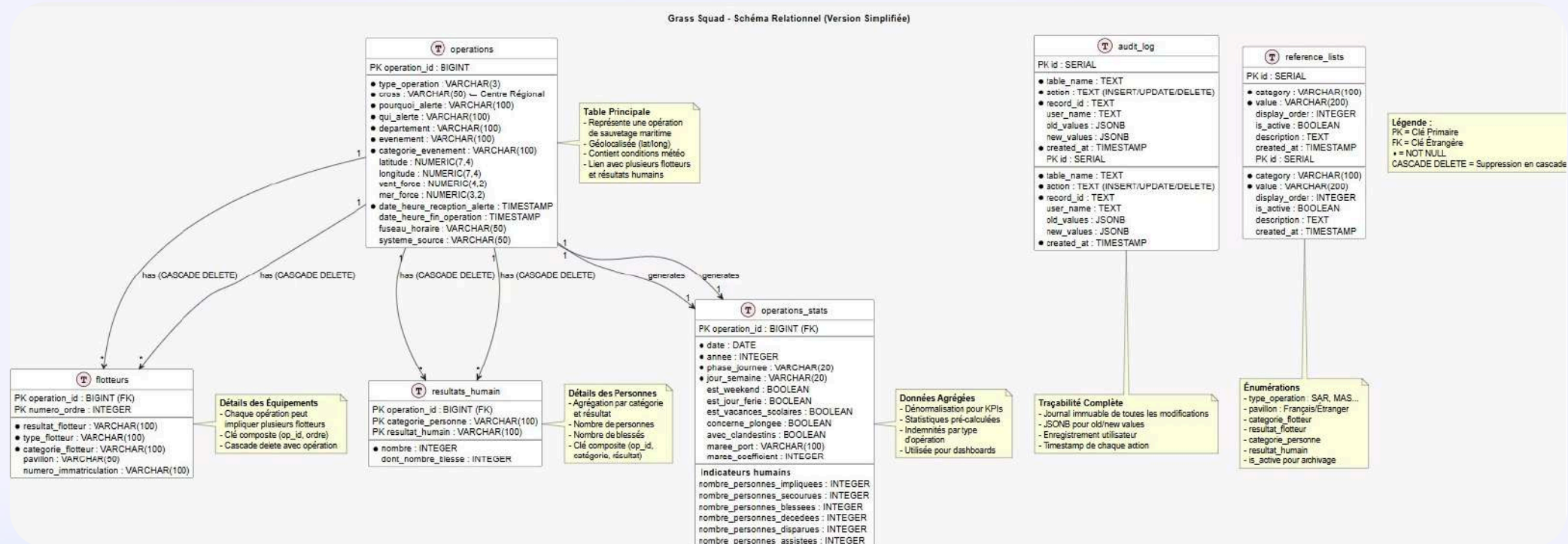
operations.csv / operations_stats.csv / flotteurs.csv / resultats_humain.csv



Les entités conceptuelles issues des fichiers CSV :

- OPÉRATION
- FLOTTEUR
- RÉSULTAT HUMAIN
- STATISTIQUES OPÉRATIONNELLE

Modèle logique de données



Points forts de cette architecture

- Normalisation : Pas de redondance inutile (sauf operations_stats pour perf)
- Intégrité : Contraintes FK et CASCADE DELETE
- Traçabilité : Chaque modification enregistrée en audit_log
- Performance : Indexes sur colonnes critiques + dénormalisation pour stats
- Flexibilité : Énumérations évolutives sans déploiement code
- Évolutivité : Structure prête pour de nouveaux champs/tables

Contraintes d'intégrité

Intégrité référentielle :

- Impossible d'avoir un flotteur sans opération
- Impossible d'avoir un bilan humain sans opération

Contraintes d'unicité :

- Un flotteur est unique par (operation_id, numero_ordre)
- Un bilan humain est unique par catégorie + résultat

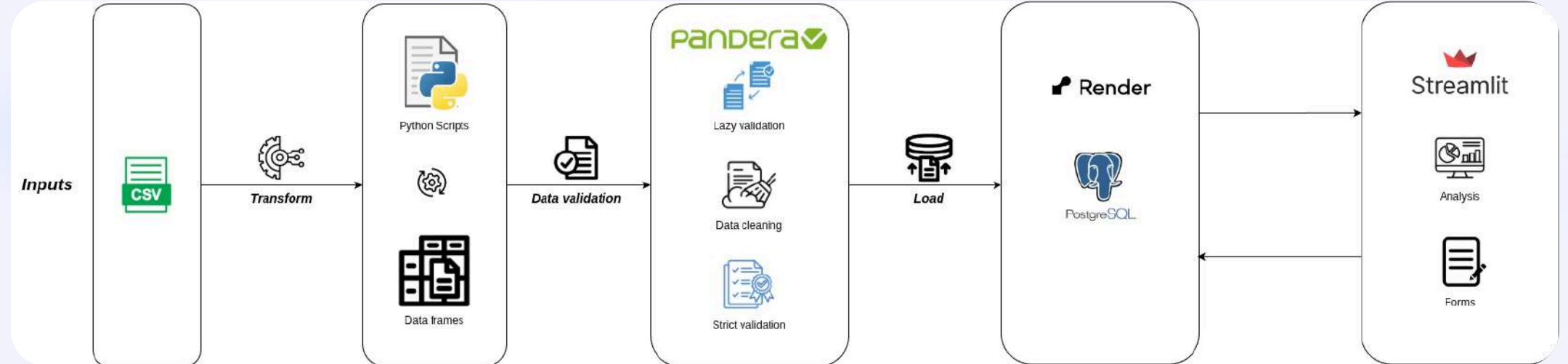
Cohérence métier

- date_fin >= date_debut
- Totaux stats cohérents avec résultats humains

03 : Pipeline ETL

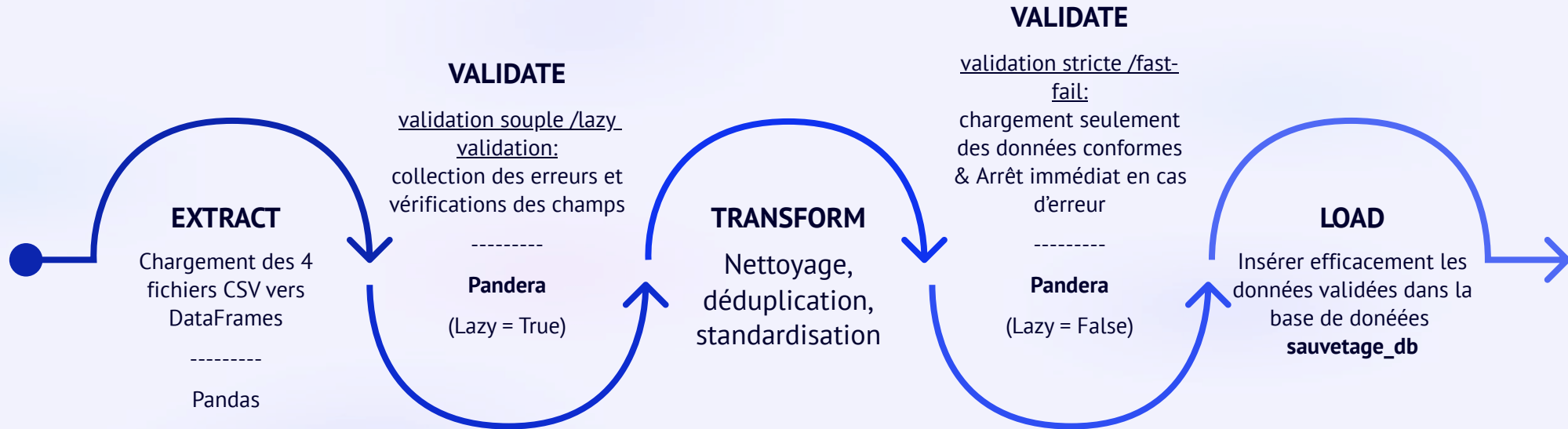


Pipeline ETL

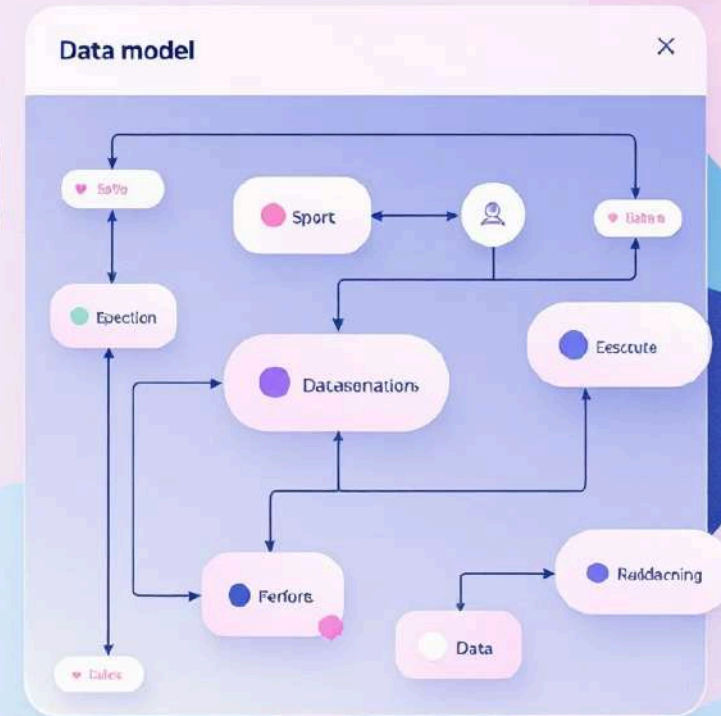


Pipeline ETL

Ingestion(Bronze) => Validation1 et transformation (Silver) => Validation stricte (Gold) => Chargement en base (Load)



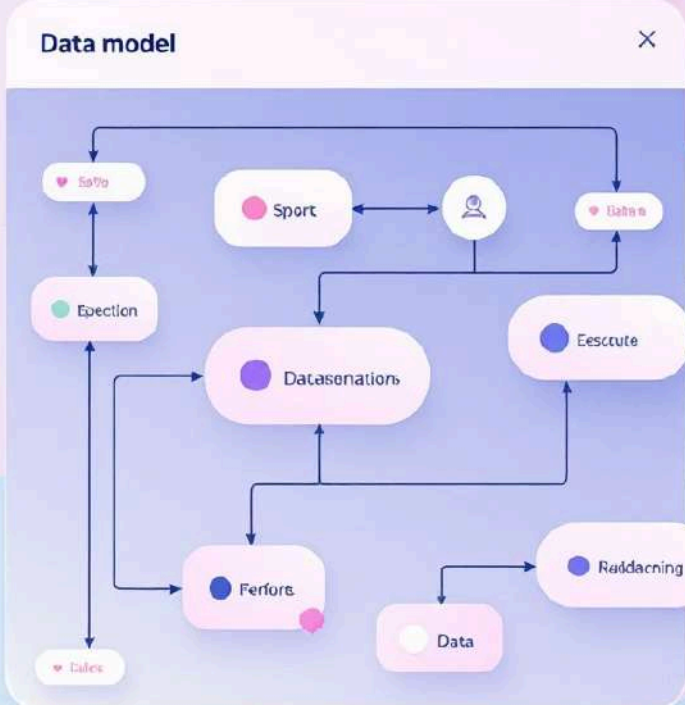
04 : Ingestion



Stratégie d'Ingestion(bronze)

- **Centralisation des sources** : chargement des quatre fichiers CSV bruts depuis un répertoire unique.
- **Lecture homogène des données** : utilisation d'une fonction générique pour standardiser l'import des fichiers (CSV ,API , Parquet).
- **Portabilité du code** : gestion dynamique des chemins afin de garantir l'exécution sur différents environnements.
- **Structuration des données** : regroupement des jeux de données dans **un dictionnaire de DataFrames** pandas.
- Préparation au pipeline ETL : données prêtes pour les étapes de nettoyage, validation et transformation.
- Aucune transformation ni correction à ce stade.

05 : Nettoyage et Validation (Silver)



Nettoyage et Validation : Pandera

Stratégie de validation : Validation en deux passes en utilisant Pandera

Pandera est une bibliothèque Python dédiée à la validation de données tabulaires (DataFrames pandas). Elle permet de définir un contrat de données (data contract) sous forme de schémas déclaratifs. L'objectif est de garantir la qualité, la cohérence et la conformité métier des données.

Pourquoi deux niveaux de validation ?

une validation stricte dès l'ingestion aurait : interrompu le pipeline & masqué les problèmes de qualité de données.

Solution :

validation souple (Lazy validation) :

Identifier toutes les erreurs sans bloquer le pipeline.

- Lazy = True
- toutes les erreurs sont collectées.
- les lignes non conformes sont extraites dans data/rejected.
- les données exploitables continuent le pipeline.
- chaque erreur est associée à une règle violée
- transparence totale sur la qualité des données sources

Nettoyage :

Après la validation souple , Les principaux transformations :

- normalisation des chaînes de caractères,
- suppression des doublons,
- correction des formats,
- filtrage des valeurs hors domaine.
- **filtrage des operation_id negatifs**

Chaque table dispose de sa fonction dédiée :

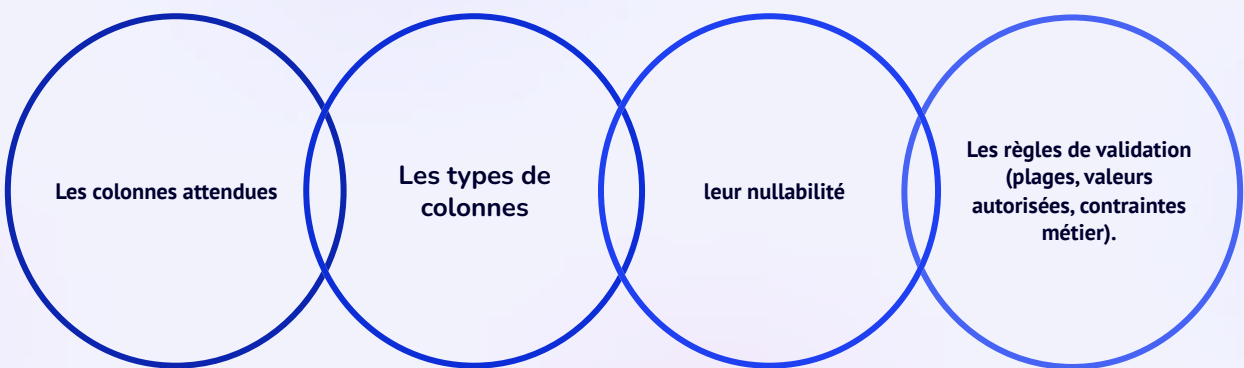
- clean_operations
- clean_operations_stats
- clean_flotteurs
- clean_resultats_humain

validation stricte (Fast-fail) :

la moindre erreur bloque le chargement.

- Lazy = False
- respect strict du schéma.
- seules les données parfaitement conformes sont conservées.
- données prêtes pour l'insertion en base PostgreSQL.

Un schéma Pandera définit :



Dans notre projet nous avons défini les schémas suivantes :

- schema_operations
- schema_operations_stats
- schema_flotteurs
- schema_resultats_humain

Nettoyage et Validation : Pandera

Exemples de règles métiers implémentées

schéma flotteurs :

```
# -----  
# Schéma pour df_flotteurs  
# -----  
schema_flotteurs = pa.DataFrameSchema({  
    "operation_id": Column(int, nullable=False),  
    "numero_ordre": Column(int, Check.ge(1), nullable=False),  
    "pavillon": Column(str, Check.isin(["Français", "Étranger"]), nullable=True),  
    "resultat_flotteur": Column(str, nullable=False),  
    "type_flotteur": Column(str, nullable=False),  
    "categorie_flotteur": Column(str, nullable=False),  
    "numero_immatriculation": Column(str, nullable=True)  
})
```

- **Intégrité référentielle** : chaque flotteur doit être lié à une opération existante.
- **Cohérence métier** :
 - numero_ordre ≥ 1
 - pavillon limité à des valeurs connues
- **Champs critiques obligatoires** :
 - resultat_flotteur
 - type_flotteur
 - categorie_flotteur

Validation souple :

- détection des doublons,
- numero_ordre invalide,
- pavillon inconnu,
- flotteurs sans opération.

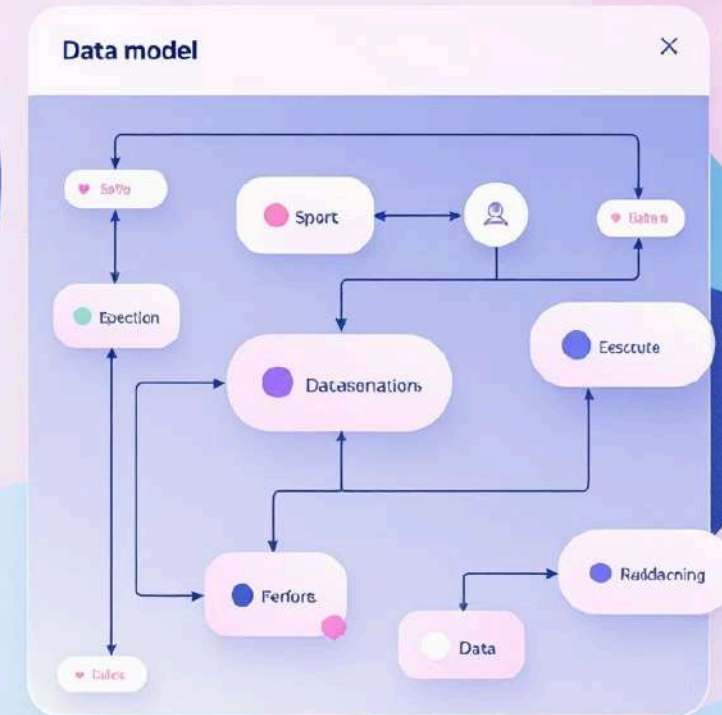
Nettoyage :

- clean_flotteurs

Validation stricte (CLEAN)

- aucun flotteur sans operation_id,
- types et valeurs conformes,
- données prêtes pour PostgreSQL.

06 : Chargement en BD



Chargement en Base de Données

Pourquoi choisir un SGBDR? (PostgreSQL)

Intégrité et cohérence des données

- Les tables sont fortement liées par operation_id (operations, flotteurs, resultats_humain, operations_stats)
- Les clés primaires et clés étrangères assurent la cohérence entre les tables
- Les contraintes permettent d'éviter les données invalides

performance sur des volumes moyens

- Les index accélèrent les recherches et les filtres sur des colonnes importantes (operation_id, date, cross , evenement ...)
- Gestion efficace des opérations **CRUD** (Create, Read, Update, Delete)

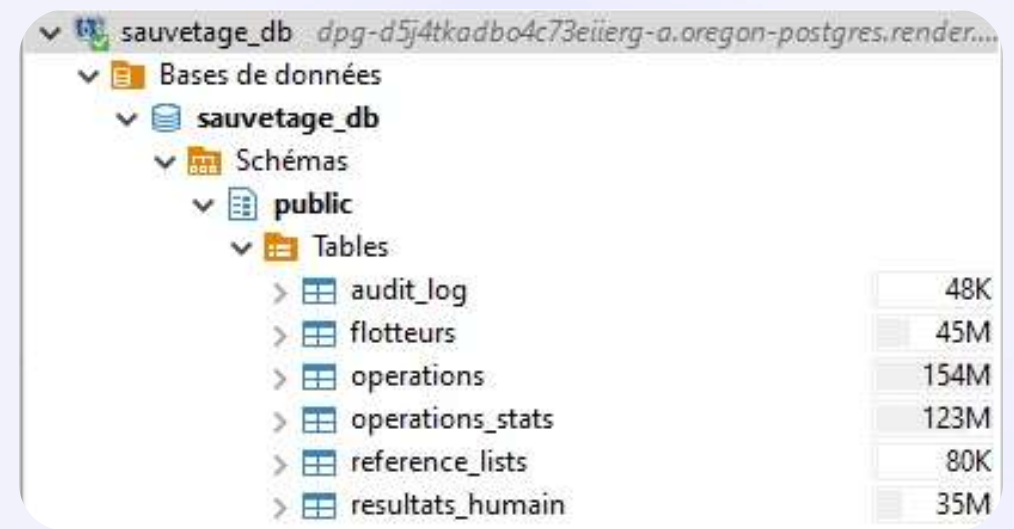
Cette approche garantit l'intégrité des données tout en maintenant des performances optimales pour l'exploitation.

Requêtes analytiques faciles

- SQL permet d'extraire rapidement des statistiques, filtrer, grouper, ou faire des jointures complexes
- Idéal pour créer des KPI métiers et alimenter les dashboards

Standard et maintenable

- SQL est un **standard reconnu**, facile à intégrer avec Python (pandas, SQLAlchemy, psycopg2)
- Permet à plusieurs développeurs de travailler sur la base sans conflits
- Compatible avec Streamlit, ETL



Exemple requête analytique

Architecture des requêtes SQL & séparation des responsabilités

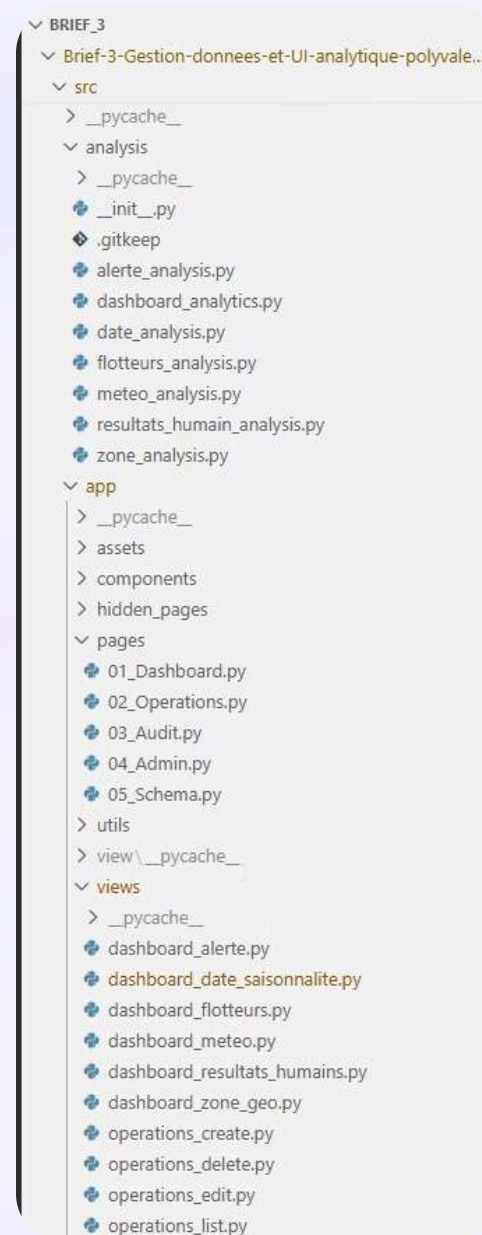
Architecture :

analysis →

- logique d'accès aux données (SQL)

views →

- la logique d'affichage (Streamlit)



← app/pages

- la logique d'affichage (Streamlit)

Séparation :

- la **logique d'accès aux données** (SQL)

> __pycache__

analysis

> __pycache__

__init__.py

.gitkeep

alerte_analysis.py

dashboard_analytics.py

date_analysis.py

flotteurs_analysis.py

meteo_analysis.py

resultats_humain_analysis.py

zone_analysis.py

app

> __pycache__

assets

components

hidden_pages

pages

01_Dashboard.py

02_Operations.py

03_Audit.py

04_Admin.py

```
9 from sqlalchemy import Engine
10
11
12 @st.cache_data(ttl=300) # Cache pendant 5 minutes
13 def get_global_kpis(_engine: Engine) -> pd.Series:
14     """
15     Récupère les KPIs globaux (opérations, personnes sauvées, décès, etc.)
16     Optimisé avec cache pour éviter les requêtes répétées.
17
18     Args:
19         _engine: Connexion à la base (préfixé _ pour ignorer dans le cache)
20
21     Returns:
22         pd.Series: KPIs avec colonnes total_secourus, total_deces, total_impliques, total_operations
23     """
24     query = """
25     SELECT
26         COALESCE(SUM(nombre_personnes_secourues), 0) as total_secourus,
27         COALESCE(SUM(nombre_personnes_tous_deces), 0) as total_deces,
28         COALESCE(SUM(nombre_personnes_impliquees), 0) as total_impliques,
29         COUNT(DISTINCT operation_id) as total_operations
30     FROM operations_stats
31     """
32     df = pd.read_sql(query, _engine)
33     return df.iloc[0] if not df.empty else pd.Series()
```

- la **logique d'affichage** (Streamlit)

app

> __pycache__

assets

components

hidden_pages

pages

01_Dashboard.py

02_Operations.py

03_Audit.py

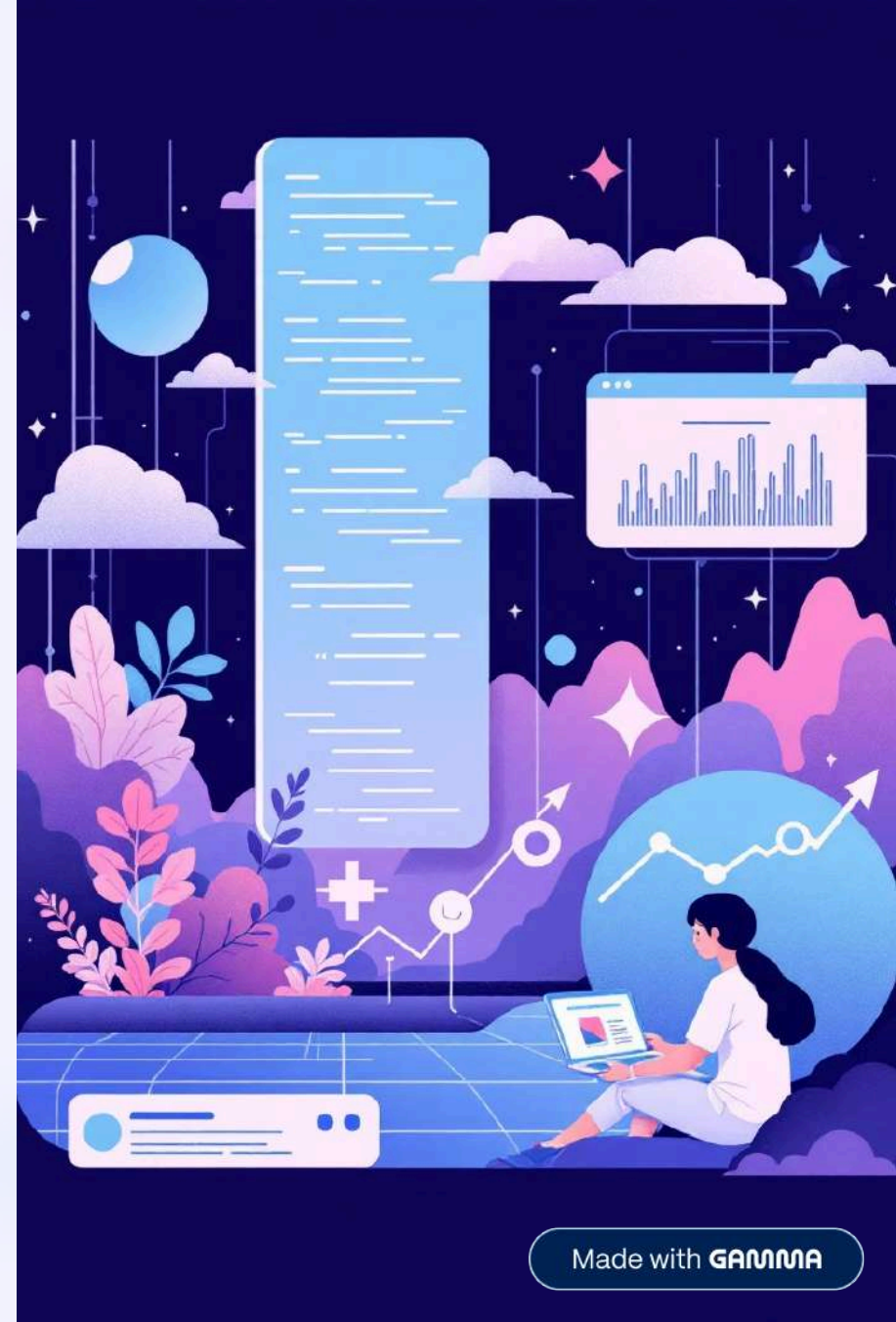
```
110 st.title("📊 Dashboard Opérationnel")
111
112 try:
113     db_info = get_database_info(engine)
114     st.success(f"✅ Connecté à la base : {db_info['current_database']}[0]}")
115
116     st.subheader("📈 Indicateurs Clés de Performance (KPIs)")
117
118     df_stats = get_global_kpis(engine)
119
```



Avantages de cette approche

- Lisibilité**
Code organisé et transparent : chaque analyse a sa place, facile à comprendre et à naviguer
- Réutilisabilité**
Écrivez une fois, utilisez partout : les requêtes deviennent des briques réutilisables sans duplication
- Maintenance facilitée**
Modifier une requête sans crainte : les changements restent isolés et n'impactent pas l'interface
- Testabilité**
Testez avec confiance : chaque fonction SQL peut être validée indépendamment
- Performance maîtrisée**
Optimisation intelligente : cache appliqué au niveau des analyses pour des résultats instantanés

07 : Tests unitaires



Test unitaires

Plusieurs tests unitaires ont été réalisés pour garantir la qualité, l'intégrité et la fiabilité des données

Ces tests concernent principalement :

Tests d'ingestion (load_raw_data.py)

- Tous les CSV chargés → 4 DataFrames
- Tables non vides et colonnes attendues présentes
- operation_id présent dans chaque table
- Vérifie la cohérence entre les tables

Tests de nettoyage (cleaning/transformation.py)

- Vérif. que chaque fonction retourne un DataFrame
- Colonnes essentielles conservées (operation_id, etc.)
- Espaces superflus retirés dans les textes
- Lignes invalides ou anomalies supprimées
- Données valides préservées

Tests de validation (validation/run_validation.py + Pandera)

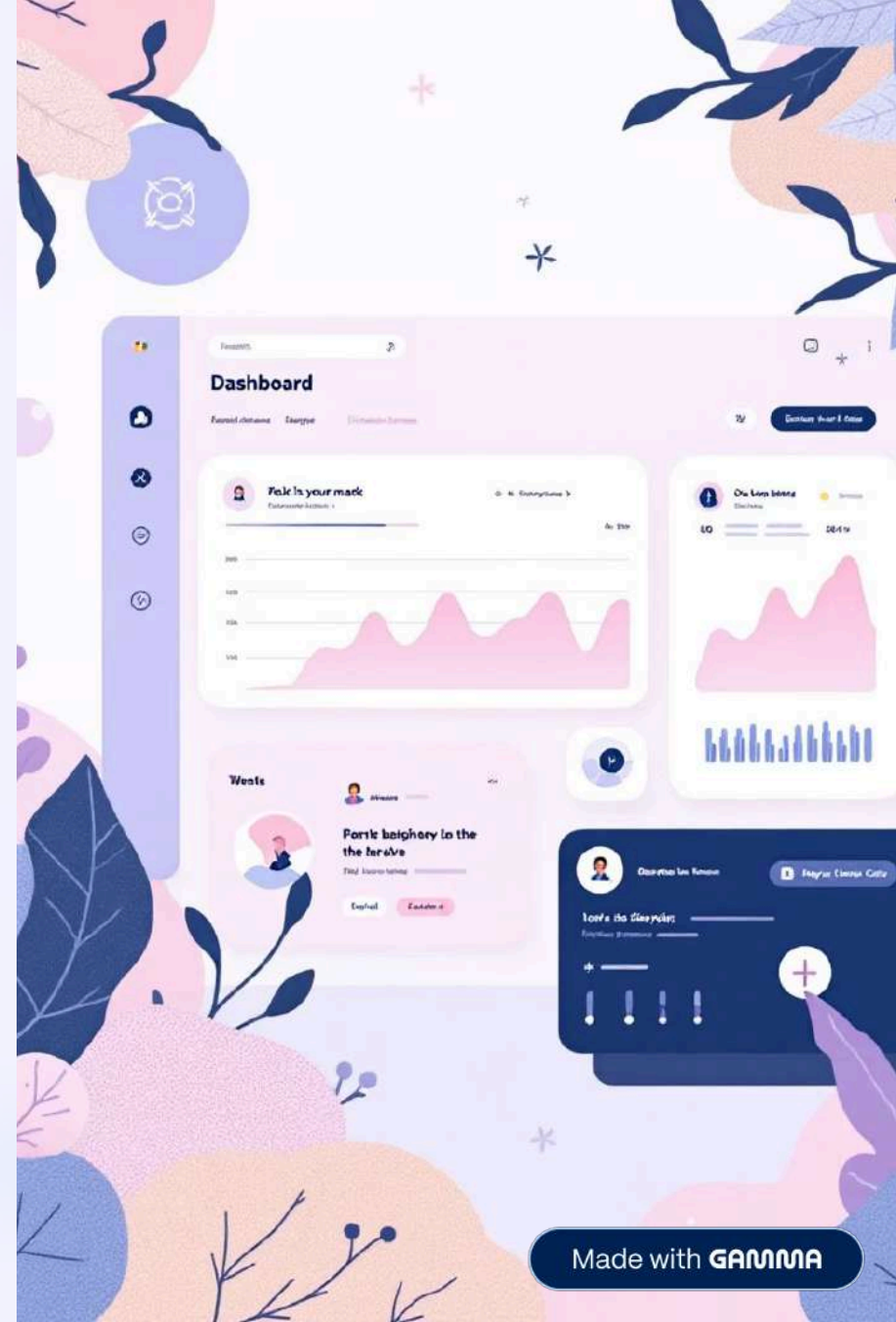
- Séparation : **curated** (valide) / **rejected** (invalide)
- Schémas respectés pour chaque table
- Gestion des cas particuliers (DataFrames vides, fichiers rejetés)

```

===== tests coverage =====
coverage: platform linux, python 3.13.9-final-0
Name                               Stmts  Miss  Cover   Missing
-----
src/cleaning/transform.py           3      3    0%    1-5
src/cleaning/transformation.py     172     25   85%    74-77, 116, 150-152, 158-160, 216-231, 236-241, 267
src/validation/_init_.py            0      0  100%
src/validation/run_validation.py    40      3   92%    42, 51, 85
src/validation/schemas_validation.py 8      0  100%
-----
TOTAL                             223     31   86%
===== 32 passed in 4.19s =====

```

08 : Mise en place de l'application streamlit



L'application Streamlit

L'application Streamlit est une **interface web interactive** pour la gestion et l'analyse des opérations maritimes. Elle permet de :

- Visualiser des tableaux de bord analytiques
- Gérer les opérations (CRUD complet create/read/update/ delete)
- Auditer les actions utilisateurs
- Administrer les listes de référence
- Analyser les données via des KPIs et graphiques

L'application est construite avec Streamlit, un framework Python qui transforme des scripts Python en applications web interactives sans nécessiter de connaissances en développement web front-end.

L'architecture de l'application & interactions

Notre application suit une architecture Model-View-Controller (MVC) adaptée à Streamlit. Ce modèle sépare le code en trois parties :

Model(Données)

Responsable de la gestion des données et de la base de données.

- Fichiers CRUD : Opérations de base de données (Create, Read, Update, Delete)
- Fichiers de base de données : Configuration et initialisation

Le Model est isolé de l'interface utilisateur et ne sait rien de Streamlit.

View(Présentation)

Gère l'affichage des données et l'interface utilisateur.

- Pages principales : Points d'entrée de l'application (home.py , pages/01_Dashboard.py ...)
- Vues métier : Logique de présentation spécifique (src/app/views/ ..)
- Composants réutilisables : Briques d'interface (src/app/components/ ...)

La View utilise Streamlit (st) pour créer l'interface et interagir avec l'utilisateur.

Controller(Logique métier)

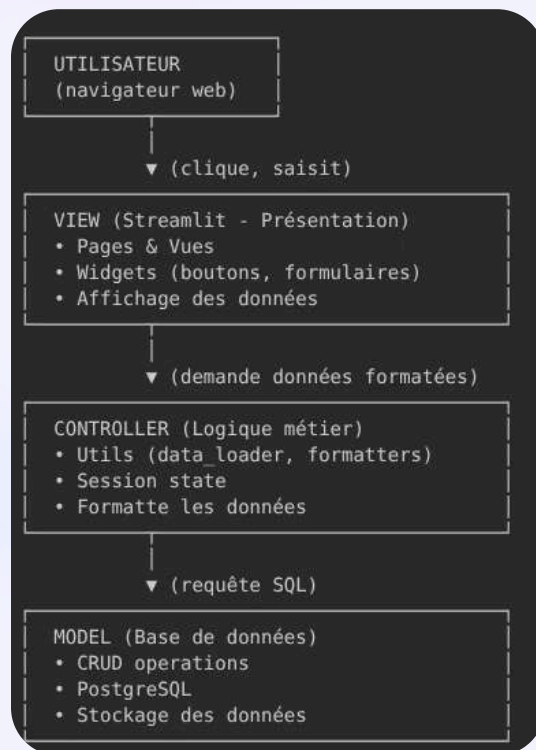
Coordonne la communication entre View et Mode

- **Utilitaires de données : Chargement et formatting**
 - src/app/utils/data_loader.py → Récupère les données (avec cache)
 - src/app/utils/formatters.py → Formate dates, nombres, texte
 - src/app/utils/queries.py → Construit les requêtes SQL
- **Gestion de l'état : Conserve les variables entre interactions**
 - src/app/utils/session_state.py → Initialise l'état de l'application

Le Controller traite les requêtes utilisateur, formate les données et met à jour l'état.

L'architecture de l'application & interactions

Schéma de flux MVC :



Avantages :

- Séparation des responsabilités
- Réutilisabilité
- Maintenabilité
- Scalabilité
- Performance

Comment Streamlit et Python interagissent ?

1. Chargement initial

- L'utilisateur accède à l'application
- Streamlit exécute le script Python de haut en bas
- Les widgets (boutons, formulaires) sont créés
- La page HTML est envoyée au navigateur

2. Interaction utilisateur

- L'utilisateur clique sur un bouton ou remplit un formulaire
- Streamlit détecte l'événement

3. Réexécution du script

- Le script Python s'exécute entièrement à nouveau
- Les variables sont recalculées
- Les données sont rechargées (sauf si en cache)

4. Session state

- `st.session_state` conserve les valeurs entre réexecutions
- Permet de maintenir l'état de l'application
- Exemple : `st.session_state.page_number = 2`

5. Affichage du résultat

- Streamlit affiche la nouvelle interface
- L'utilisateur voit les modifications

Streamlit component

- **Utilisation des composants pour :**
 - la configuration de la page
 - la mise en page
 - title / header / markdown ...
 - navigation : switch_page
 - les buttons / formulaires / inputs / selectbox / champs texte / ...
 - message : error, success , warning
 - les dataframes
 - session (session_state)
 - Cache (@st.cache_data)
 - les graphiques
 - les KPIs (métriques)

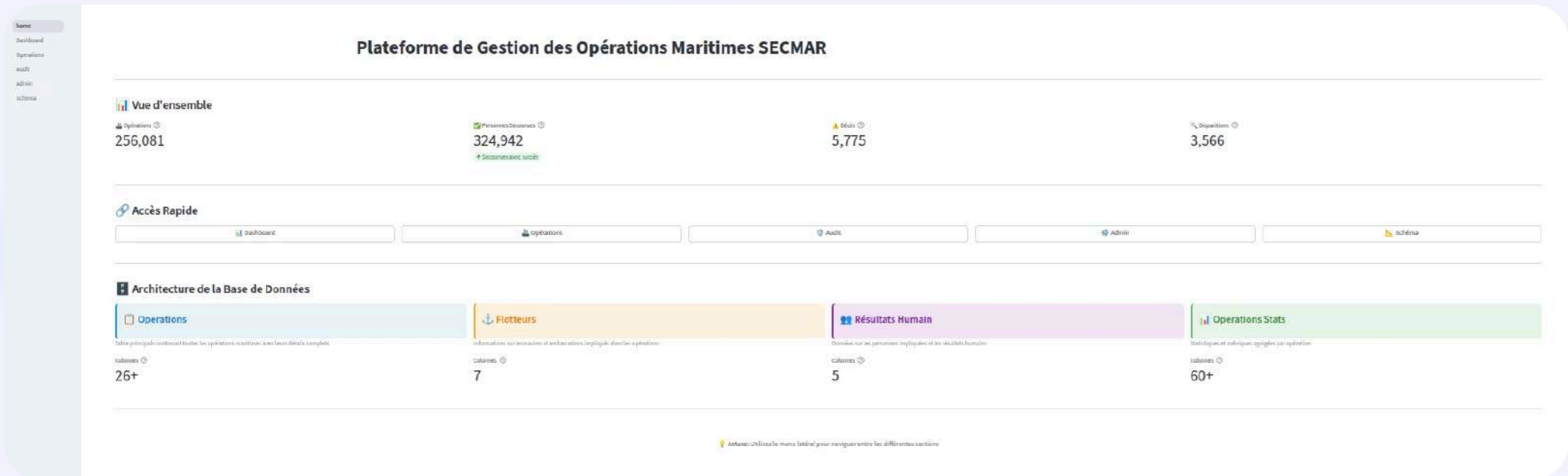
Résumé des composants clés utilisés dans le projet		
Composant	Utilisation	Fichier
<code>st.set_page_config</code>	Configuration globale	Toutes les pages
<code>st.columns</code>	Layout en colonnes	<code>home.py</code> , <code>operations_list.py</code>
<code>st.metric</code>	Affichage KPIs	<code>home.py</code> , <code>dashboards</code>
<code>st.dataframe</code>	Tables interactives	<code>operations_list.py</code>
<code>st.button</code>	Actions utilisateur	Partout
<code>st.selectbox</code>	Sélection unique	Formulaires
<code>st.form</code>	Groupement inputs	<code>operations_create.py</code>
<code>st.session_state</code>	Gestion d'état	Partout
<code>@st.cache_data</code>	Cache données	<code>data_loader.py</code>
<code>st.rerun</code>	Rechargement page	Après actions
<code>st.switch_page</code>	Navigation	<code>home.py</code>
<code>st.spinner</code>	Indicateur chargement	Vues avec requêtes
<code>st.bar_chart</code>	Graphiques barres	Dashboards
<code>st.map</code>	Cartes géographiques	<code>dashboard_zone_geo.py</code>

09 : Demonstration



Home





Dashboards & visualisations



[Retour au dashboard](#)

Dashboard Opérationnel

✓ Connecté à la base : sauvegarde_db

Indicateurs Clés de Performance (KPIs)

📊 Total opérations

256,080

👤 Personnes impliquées

782,310

📄 Personnes déguisées

324,942

📈 Taux de réussite

41.54%

Carte des Opérations



Explorer par dimensions

[Retour au dashboard](#)

Météo — Vent, mer et lien avec la gravité

Objectif : identifier si la météo augmente l'activité et si elle rend les opérations plus "sérieuses" (humaines).

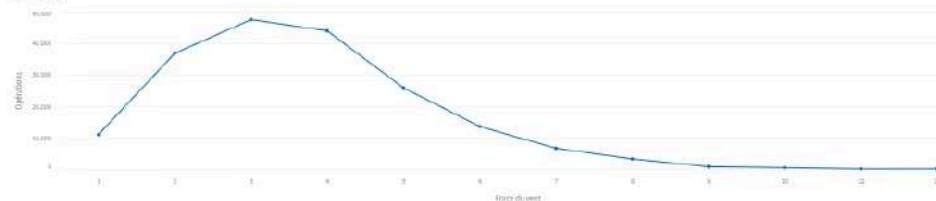
? Questions métier

- Les opérations augmentent-elles avec la force du vent / la force de la mer ?
- Les conditions dégradées sont-elles associées à plus de personnes impliquées ?
- Existe-t-il un seuil à partir duquel la gravité augmente nettement ?
- Les marées influencent-elles le volume d'opérations et la charge humaine ?

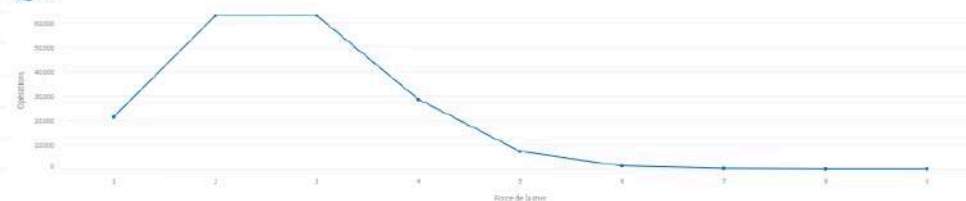
🌊 Volume d'opérations selon le vent et la mer

Lecture : comparez rapidement l'évolution du volume selon la force du vent (gauche) et de la mer (droite).

🌊 Vent



🌊 Mer

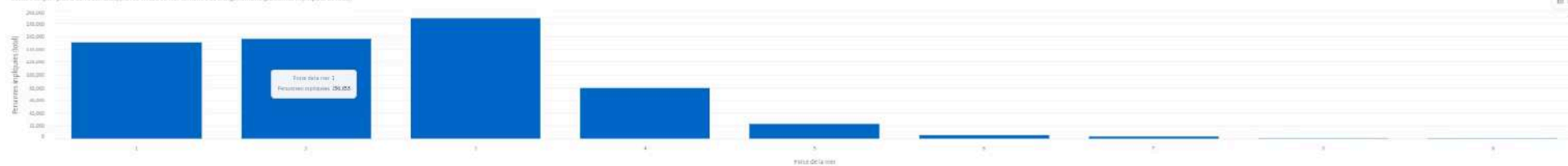


📊 Plateaux (stabilité)

📊 Résultats humains

👤 Gravité humaine — Personnes impliquées selon la force de la mer

Lecture simple : plus la barre est haute, plus ce niveau de mer implique de charge humaine (personnes impliquées au total).



✔ **Lecture simple :** la force de mer qui concentre le plus de charge humaine est 3 avec 189,045 personnes impliquées (33.2% du total).

Répartition des opérations selon la mer

Retour au dashboard

Date — Saison, vacances et jours à risque

Objectif : l'objectif est de savoir à quels moments de l'année et de la semaine les opérations sont les plus nombreuses, et si certaines périodes sont aussi plus graves en termes de personnes impliquées.

? Questions métier

- Y a-t-il plus d'opérations pendant les vacances scolaires ?
- Y a-t-il plus d'opérations en été (juillet-août) ?
- Quels jours de la semaine concentrent le plus d'activité et de charge humaine ?
- Les périodes "à risque" sont-elles plus graves (plus de personnes impliquées par opération) ?

+ Vue globale

Opérations
256,080

Personnes impliquées
782,310

Séances
324,942
+ 41.54%

Part des personnes séancées parmi toutes les personnes impliquées 41.54%

Incidents
5,775
+ 0.74%

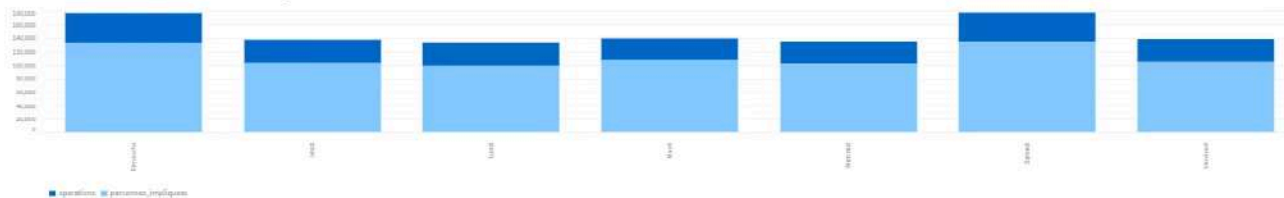
Part des personnes décelées parmi toutes les personnes impliquées 0.74%

Impliqués / opération
3.05

Lecture simple

- Opérations = le volume d'activité.
- Personnes impliquées = la charge humaine totale.
- Impliqués / opération = une mesure simple de "gravité moyenne".

17 Jour de la semaine — activité & charge



	jour_semaine	operations	personnes_impliquees	impliquees_par_operation
0	Dimanche	145122	153122	2.03
1	Lundi	117032	117032	2.97
2	Mardi	117032	117032	3.09
3	Mercredi	117032	117032	3.36
4	Jeudi	117032	117032	2.96
5	Vendredi	117032	117032	3.07
6	Samedi	117032	117032	3.16

Analyse automatique

- Jour avec le plus d'opérations : Dimanche (145,122 opérations).
- Jour avec la plus forte charge humaine : Samedi (124,311 personnes impliquées). 🟡 Cela aide à planifier des moyens renforcés sur les jours les plus chargés.

Vacances scolaires vs Hors vacances



	période	operations	personnes_impliquees	impliquees_par_operation
0	Non-vacances	2170320	511680	2.96
1	Vacances scolaires	2140310	713110	3.12

Crud & interactions avec la base de données

Opérations



Gestion des Opérations

Liste des Opérations

[Créer une nouvelle opération](#)

operation_id

Trier par

date alerte

Ordre

DESC

ASC

Éléments par page

10

Tableau des Opérations

ID	CROISS	Événement	Date/Heure	Dépt	Coordonnées	Actions
110420241728	Bel		2020-01-16 15:03			  
1412022002	Bel	Transport sanitaire...	2020-12-01 11:29	Modulun	47.66, -1.45	  
1112022494	Corse	Explosion de mouillage	2020-12-01 17:18	Finistère	50.05, 9.89	  
11032021454	Polynésie	Situation indétectable...	2020-12-01 09:43	Polynésie	-19.76, -149.14	  
11032022433	Corse	Engin dangereux à bord...	2020-12-01 09:35	Finistère	48.11, -4.53	  
11032022432	Corse	Situation indétectable...	2020-12-01 09:23	Finistère	48.10, -4.52	  
11032021437	Polynésie	Situation indétectable...	2020-12-01 08:10	Polynésie	-17.43, -149.11	  
11032021502	Bel	Accident électrique	2020-12-01 06:27	Landes	44.17, -1.36	  
11032022503	Bel	Autre événement	2020-12-01 03:27	Gironde	45.36, -1.07	  
11032022501	Bel	Heurt	2020-12-01 02:18	Gironde	45.35, -1.04	  

1

2

...

20609





Total opérations
256081

CROISS le plus actif
Étel (4)

Opérations affichées
10



Gestion des Opérations



Consultation de l'Opération #110420241728

[Retour à la liste](#)

CROSS

Etel

Evénement

N/A

Département

N/A

Date

2026-01-16 15:03

Modifier

Supprimer

[Opérations](#) [Fleets](#) [Résultats Humains](#) [Statistiques](#)

Données complètes de l'opération

	Valeur
operation_id	110420241728
type_operation	MAG
pourquoi_alerte	None
moyen_alerte	None
qui_alerte	None
categorie_qoi_alerte	None
crise	DEL
departement	None
est_metropolitain	None
evenement	None

Historique d'audit de cette opération

Date	Action	Utilisateur	Détails	SQL	Anciennes valeurs	Nouvelles valeurs
2026-01-16 16:34:44	VIEW	system	Consultation détaillée de l'opération 110420241728	SELECT * FROM operations WHERE operation_id = 110420241728		
2026-01-16 16:37:57	VIEW	system	Consultation détaillée de l'opération 110420241728	SELECT * FROM operations WHERE operation_id = 110420241728		
2026-01-16 16:19:24	UPDATE	system	Modification de l'opération 110420241728 - 1 champ(s) modifié(s)	UPDATE operations SET type_operation = 'MAG', pourquoi_alerte = NULL, moyen_alerte = NULL, qui_alerte = NULL, categorie_qoi_a ['crise'] 'ok'		['crise'] 'Etel'
2026-01-16 16:12:42	UPDATE	system	Modification de l'opération 110420241728 - 1 champ(s) modifié(s)	UPDATE operations SET type_operation = 'MAG', pourquoi_alerte = NULL, moyen_alerte = NULL, qui_alerte = NULL, categorie_qoi_a ['crise'] 'testaudit'		['crise'] 'ok'
2026-01-16 16:09:47	UPDATE	system	Modification de l'opération 110420241728 - 1 champ(s) modifié(s)	UPDATE operations SET type_operation = 'MAG', pourquoi_alerte = NULL, moyen_alerte = NULL, qui_alerte = NULL, categorie_qoi_a ['crise'] 'Jobourg'		['crise'] 'testaudit'
2026-01-16 16:09:48	INSERT	system	Création opération - CROSS: Jobourg	None		['crise'] 'Jobourg', 'autorité': None, 'latitude': None, 'evenement': None
2026-01-16 16:09:45	INSERT	system	Nouvelle opération créée - CROSS: Jobourg	INSERT INTO operations (operation_id, 'crise', date_heure_reception_alerte, type_operation, pourquoi_alerte, moyen_i		['crise'] 'Jobourg', 'autorité': None, 'latitude': None, 'evenement': None

7 action(s) enregistrée(s) dans l'audit

home

Dashboard

Operations

Audit

Admin

Schema

 **Gestion des Opérations**

 **Modifier l'Opération #110420241728**

Retour à la liste

Opérations

Flotteurs

Résultats-Humains

Statistiques

Modifier les données de l'opération

Modifiez les valeurs directement dans le tableau ci-dessous

Operation ID	P_ type operation	P_ pourquoi alerte	P_ moyen alerte	P_ qui alerte	P_ catégorie qui alerte	P_ cross	P_ département	P_ est métropolitain	P_ événement	P_ catégorie événement	P_ autorité	P_ seconde autorité	P_ zone responsabilité	P_ latitude	P_ longitude	P_ vent direction	P_ vent direction catégorie	P_ vent force	P_ ray force	P_ date heure reception alerte	P_ date heure fin operation	P_ numero strip	P_ cross strip	P_ niveau securite
110420241728	REAL						FR													2024-01-10 17:13:35	2024-01-21 04:13:39			

Savegarder l'opération

Made with GAMMA

Gestion des Opérations

Supprimer l'Opération #110420241728

 Annuler et retourner à la liste

 **Attention** : Cette action est irréversible et supprimera toutes les données associées !

 Aperçu de l'opération à supprimer :

operation_id	type_operation	pourquoi_alerte	comment_alerte	qui_alerte	categorie_alerte	cxis	departement	est_metroplitain	evenement	categorie_evenement	autorite	seconde_autorite	zone_responsabilite	latitude	longitude	vent_direction	vent_direction_categorie	vent_force	mer_force	date_heure_reception_alerte	date_heure_fin_operation	numéro_sitrap	cxis_sitrap	niveau_forais	systeme_source
110420241728	IAS	None	None	None	None	PM	None	None	None	None	None	None	None	None	None	None	None	None	None	2026-01-10 15:43:19	2026-01-21 01:15:40	None	None	None	None

Cette opération contient :

- 1 fluxmar(y)
- 1 résultat(s) terrain(s)
- 0 statistique(s)

Toutes ces données seront supprimées de manière définitive.

 ☒ Je confirme vouloir supprimer cette opération et toutes ses données

 SUPPRIMER DÉFINITIVEMENT

Audit



admin



Administration - Listes de Référence

Listes de Référence Session Audit

Gestion des Listes de Référence

Initialisation des listes de référence

Cette action va exécuter le script SQL pour créer/initialement la table référence_list

Cette opération va :

- Supprimer la table référence_list si elle existe
- Créer une nouvelle table référence_list
- Insérer toutes les valeurs par défaut
- Attention : Les valeurs personnalisées seront perdues !

Initialiser les listes de référence

Sélectionnez une catégorie pour gérer ses valeurs

categorie_flotteur

Catégories de flottes (commerce, pêche, etc.)

5 valeurs

Gérer

categorie_personne

Catégories de personnes impliquées

10 valeurs

Gérer

cross

Opérations de surveillance et de sauvetage

7 valeurs

Gérer

marce_categorie

Catégories de coefficient de marée

4 valeurs

Gérer

pavillon

Pavillons des navires (français, étranger)

2 valeurs

Gérer

prefecture_maritime

Préfectures maritimes

3 valeurs

Gérer

resultat_flotteur

Résultats possibles pour les flottes

3 valeurs

Gérer

resultat_humain

Résultats humains (Séjours, événements, etc.)

3 valeurs

Gérer

systeme_source

Systèmes sources des données

2 valeurs

Gérer

type_operation

Type d'opération (SAR, MAS, etc.)

3 valeurs

Gérer

vent_direction_categorie

Directions cardinales du vent

1 valeur

Gérer

Administration - Listes de Référence

🔗 Listes de Référence 🔗 Gestion Audit

Gestion de la Table Audit

⚠️ ATTENTION: Les opérations ci-dessous sont irréversibles

Statistiques de la table audit_log

total d'enregistrements

9

Premier log

16/01/2026 14:03

Dernier log

16/01/2026 16:34

Répartition par type d'action:

Action	Nombre
INSERT	4
UPDATE	1
VIEW	2

Vider la table audit

Cette opération supprimera tous les enregistrements de la table audit_log.

- ✅ La structure de la table sera préservée
- ❌ Toutes les données d'audit seront perdues
- ⚠️ Cette action est irréversible!

Vider la table

Réinitialiser la table audit

Cette opération va:

- 🗑️ Supprimer la table audit_log existante
- 🔄 Recréer la table avec sa structure d'origine
- 🔄 Réinitialiser les index et contraintes
- ⚠️ Cette action est irréversible!

Réinitialiser

schéma

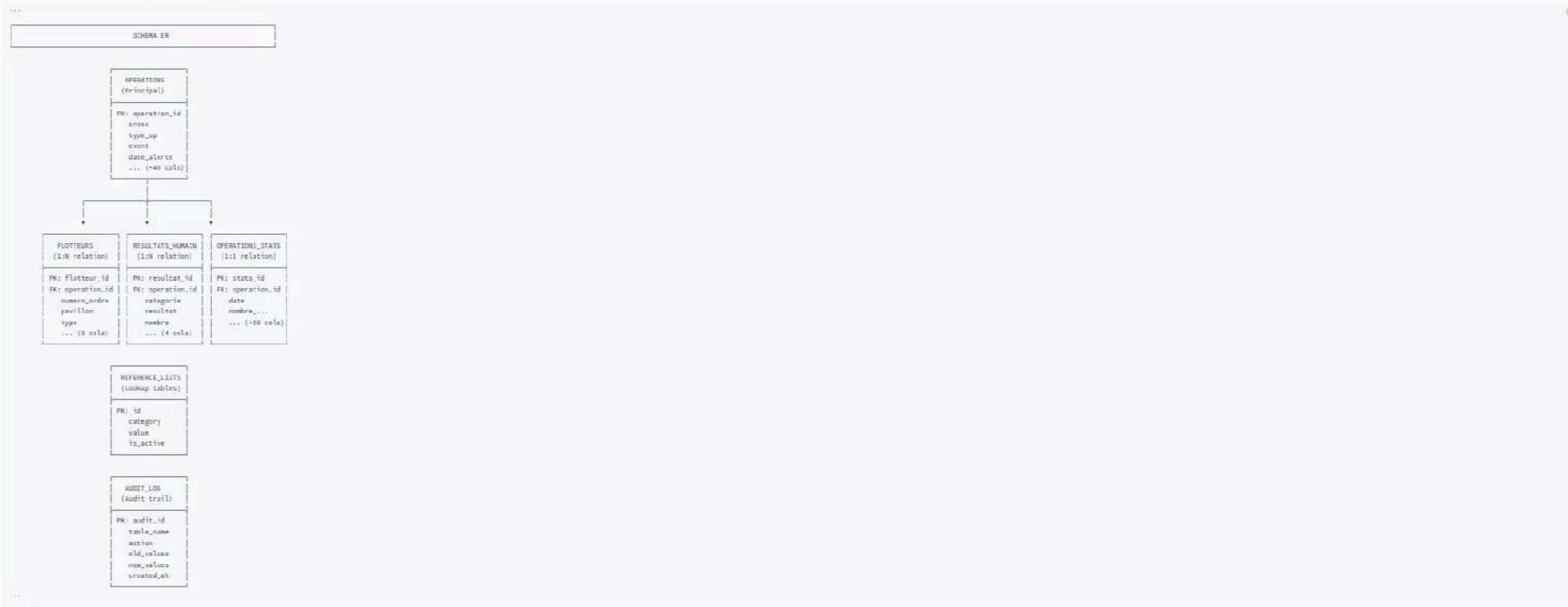


Schéma de Base de Données

Cette page affiche la structure complète de la base de données avec :

- Diagramme ER : Relations entre toutes les tables
- Détails des tables : colonnes, types, contraintes (PK/FK/NOT NULL)

Diagramme Entité-Relation (ER)



Détails des Tables

Sélectionnez une table pour voir les détails des colonnes:

Détails des Tables

Sélectionnez une table pour voir les détails des colonnes:

Choisissez une table:

OPERATIONS - Table principale - Opérations de sauvetage maritime

OPERATIONS

Table principale - Opérations de sauvetage maritime

Colonne	Type	PK	FK	Nullable	Description
operation_id	INTEGER				Identifiant unique de l'opération
coast	VARCHAR(50)				Centre régional opérationnel
type_operation	VARCHAR(100)			✓	Type d'opération (SAR, MAS, SUR, FOL, DR)
evenement	VARCHAR(100)			✓	Type d'événement
categorie_evenement	VARCHAR(100)			✓	Catégorie d'événement
date_heure_reception_alerte	TIMESTAMP				Date et heure de l'alerte
date_heure_fin_operation	TIMESTAMP			✓	Date et heure de fin
pourquoi_alerte	TEXT			✓	Raison de l'alerte
moyen_alerte	VARCHAR(100)			✓	Moyen d'alerte (VHF, téléphone, etc.)
qui_alerte	VARCHAR(100)			✓	Qui donne l'alerte

Colonnes

28

Clefs Primaires

1

Clefs Étrangères

0

Informations en Direct de la Base de Données

Nombre de lignes par table:

Table	Lignes
operations	250081
betweens	300000
resultats_humans	184670
operations_stats	334000
audit_log	5
reference_logs	65

Taille des tables:

Table	Taille
operations	215 MB
betweens	4x MB
resultats_humans	35 MB
operations_stats	175 MB
audit_log	34 KB
reference_logs	64 KB

Légende

Symboles:

- Clef Primaire (PK)
- Clef Étrangère (FK)
- Accepte les valeurs NULL

Types de données courants:

- INTEGER: Nombres entiers
- VARCHAR(n): Texte de longueur variable
- NUMERIC(m,d): Nombres décimaux
- BIGINT, VARCHAR, etc.

Types de données (suite):

- TIMESTAMP: Date et heure
- DATE: Date uniquement
- TEXT: Texte long
- JSONB: Données JSON

Note technique:

- Les relations 1:N sont implémentées via les clés étrangères (FK)
- La relation 1:1 entre operations et operations_stats est UNIQUE
- audit_log enregistre toutes les modifications avec old/new values
- reference_logs fournit les valeurs pour les déploiements

Conclusion & Perspectives

